Exploit:
Buffer Overflow, ROP, NX, CFI, Canary

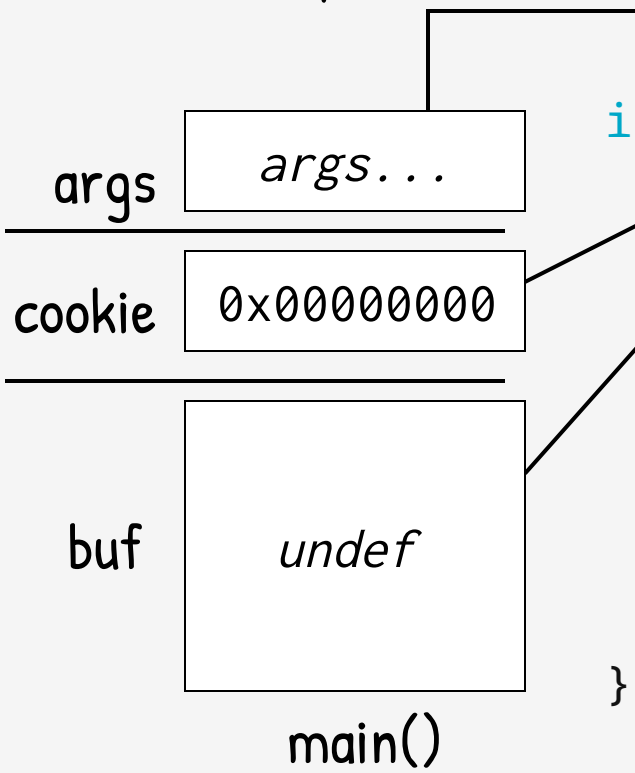
Taller de Seguridad

Ing. Martin Di Paola - Fiuba

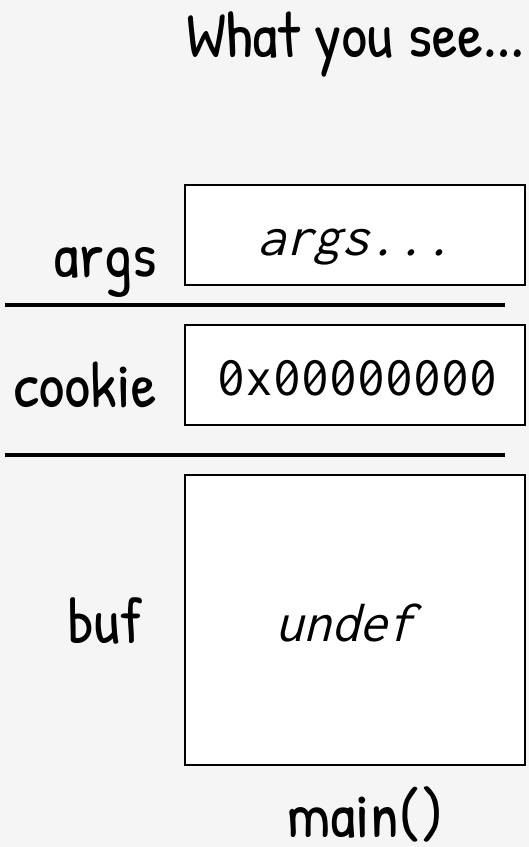
Can you print *"You win"*?

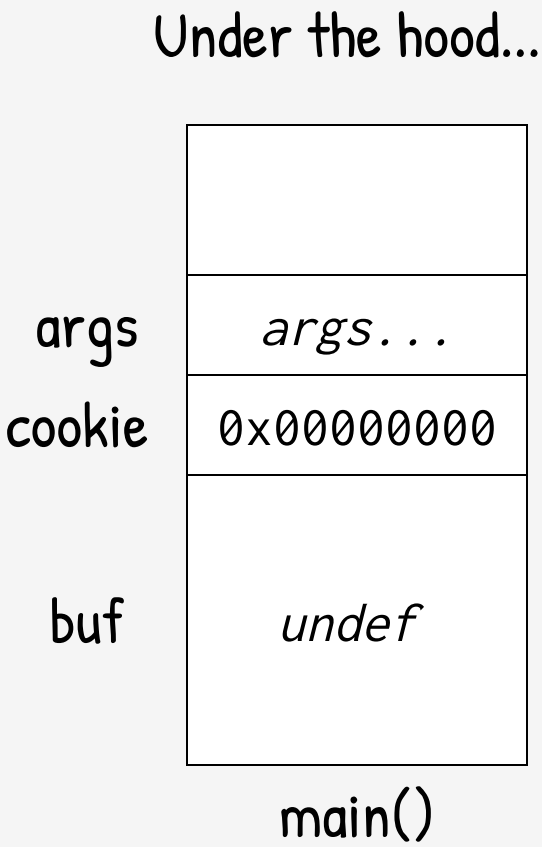
```
int main() {
    int cookie = 0x0000000;
    char buf[80];
    gets(buf);
    if (cookie == 0x41424344) {
        puts("You win.");
    }

    return 0;
}
```

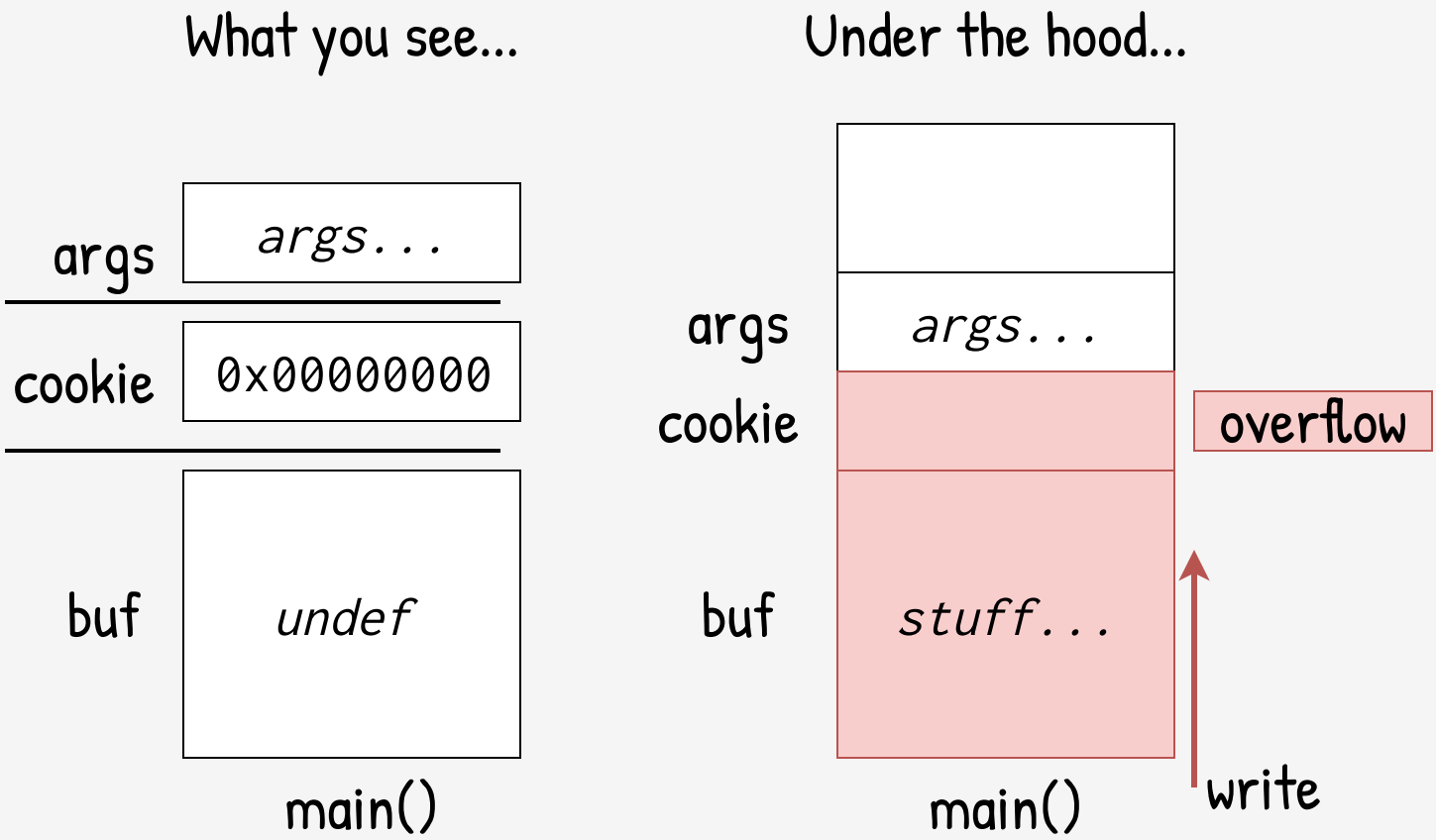
What you see...

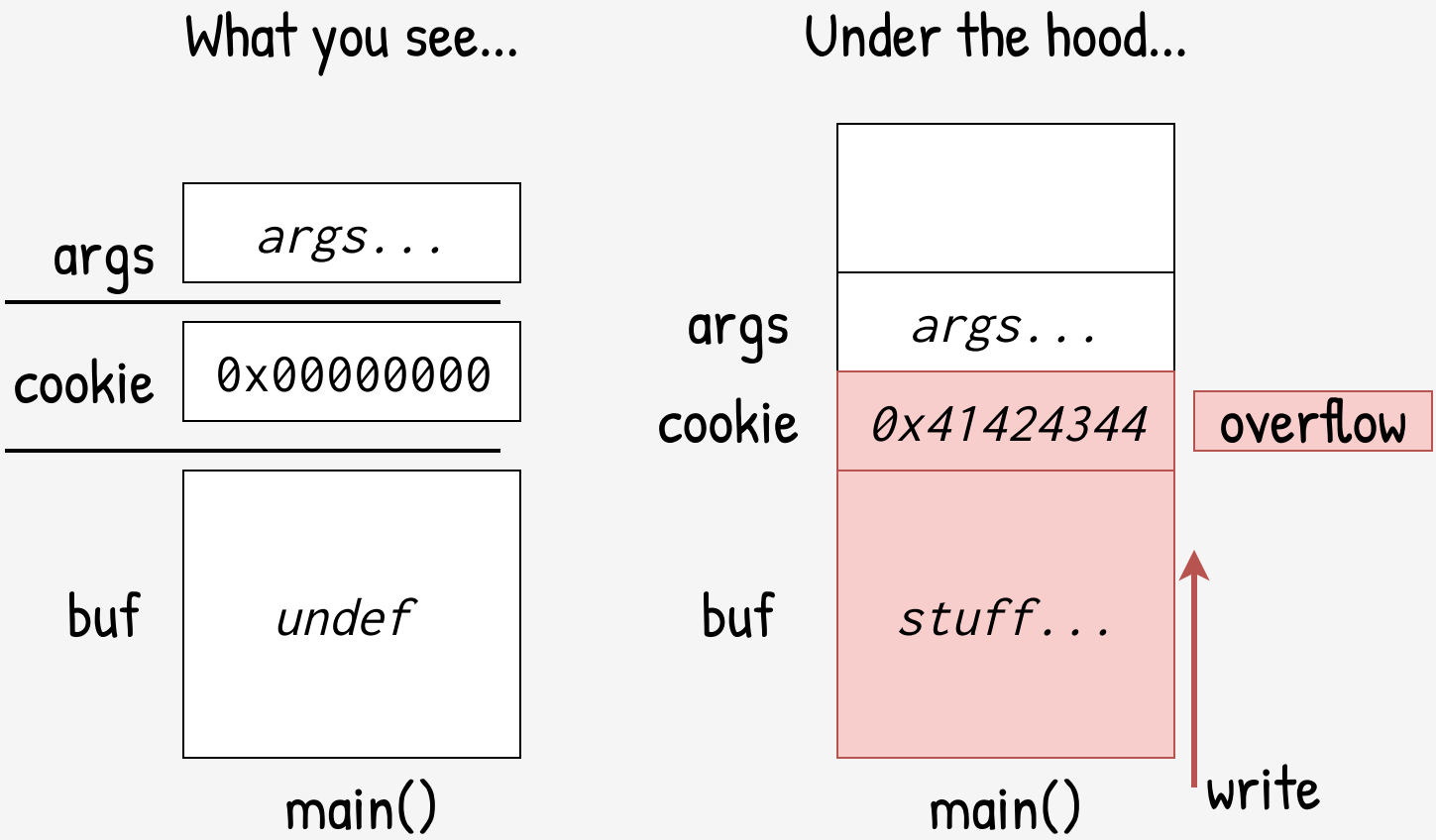



```
int main() {
    int cookie = 0x0000000;
    char buf[80];
    gets(buf);
    if (cookie == 0x41424344) {
        puts("You win.");
    }

    return 0;
}
```


What you see...
args
args...
cookie
0x00000000
buf
undef
main()


Under the hood...
args
args...
cookie
0x00000000
buf
undef
main()


What you see...
Under the hood...
args
args...
cookie
0x00000000
buf
undef
main()
args
args...
cookie
overflow
buf
stuff...
write
main()


What you see...
Under the hood...
args
args...
cookie
0x00000000
buf
undef
main()
args
args...
cookie
0x41424344
overflow
buf
stuff...
write
main()

Can you print *"You win"*?

Can you print whatever you like?

↓

Can you take control of the program?

↓

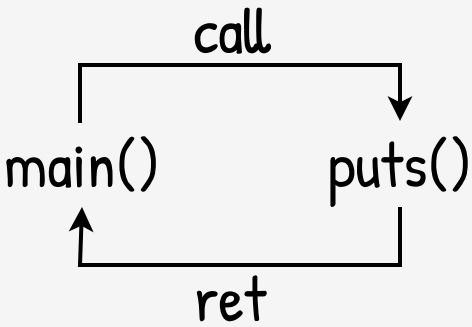
Can you take control of the host?

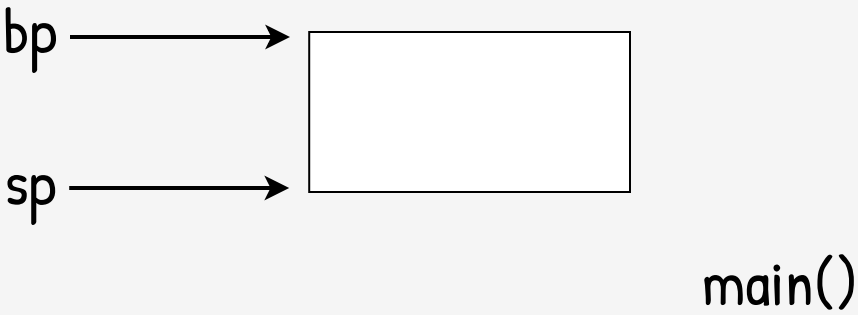
↓

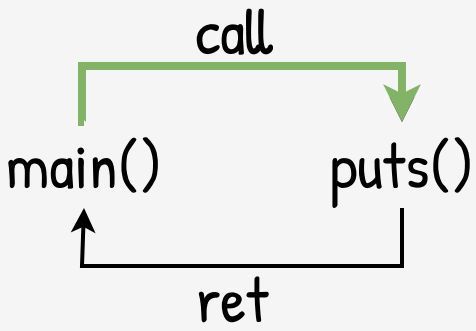
Can you ***truly win***?

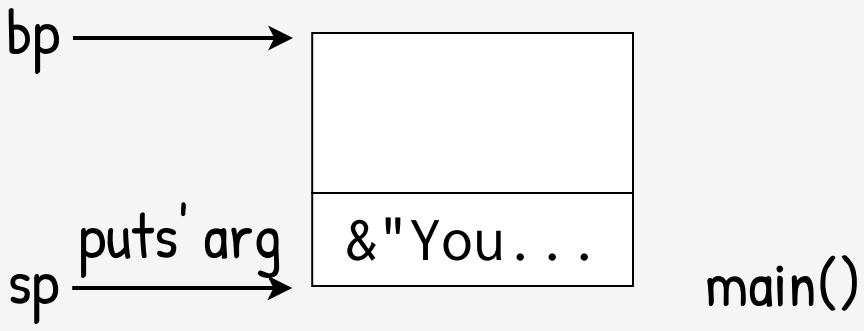
```
int main() {
    int cookie = 0x0000000;
    char buf[80];
    gets(buf);
    if (cookie == 0x41424344) {
        puts("You win.");
    }

    return 0;
}
```

```c
int main() {
    int cookie = 0x0000000;
    char buf[80];
    gets(buf);
    if (cookie == 0x41424344) {
        puts("Your flag is ~/flag.txt.");
    }

    return 0;
}
```

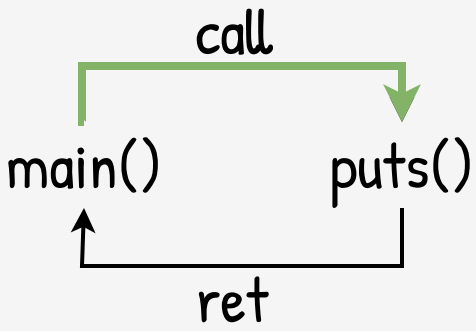

call
main()
puts()
ret

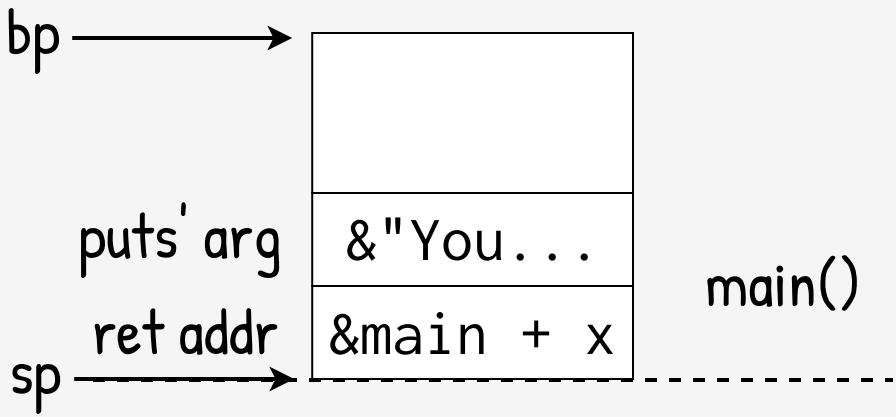

bp
sp
main()



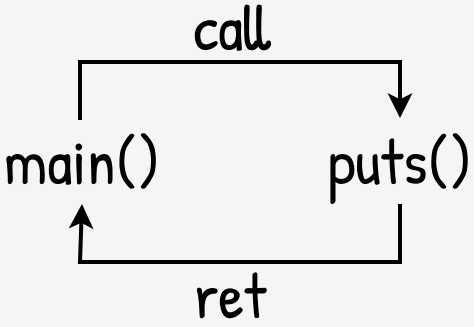


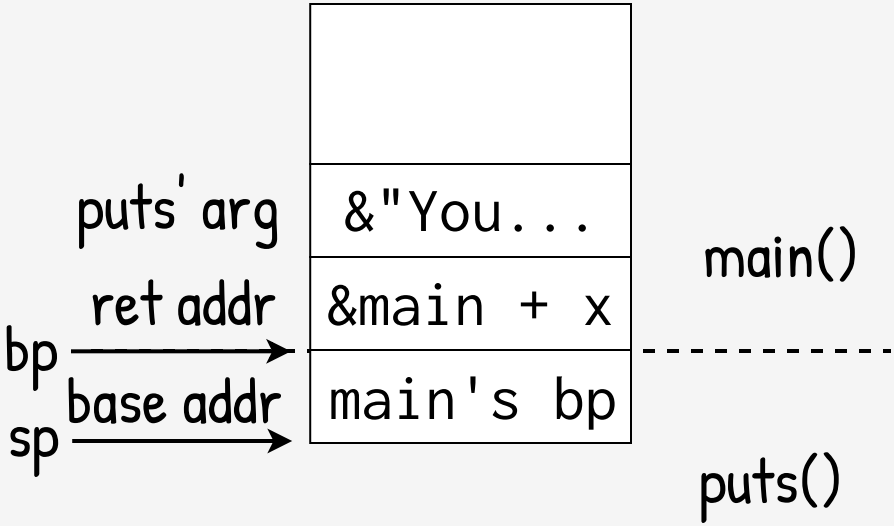

```
mov ecx &"You..."
push ecx
call &puts
```



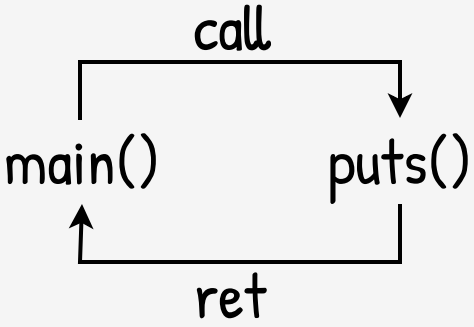


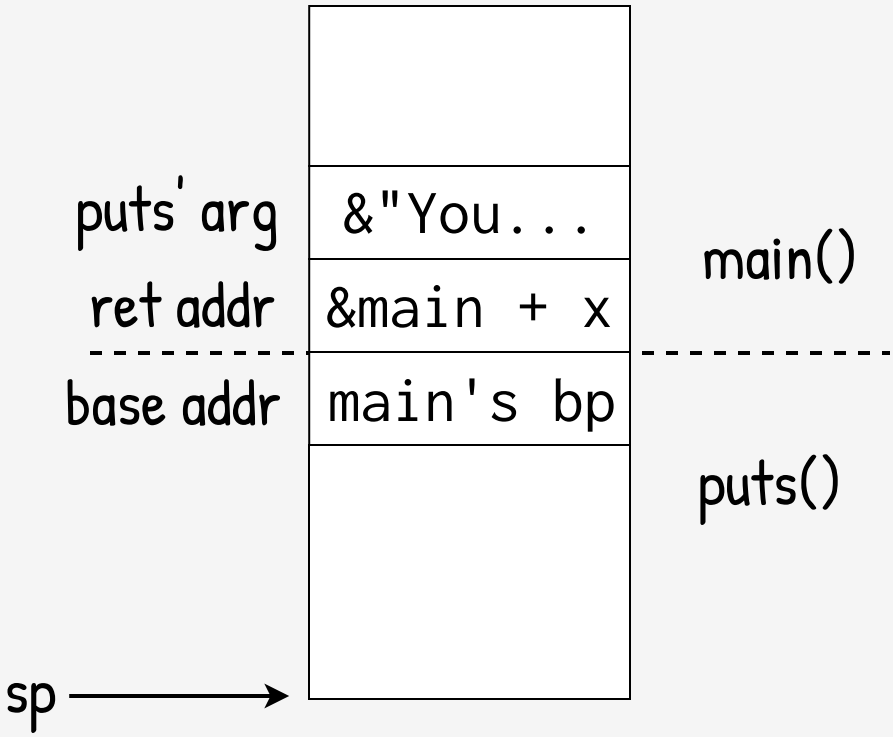

```
  mov ecx &"You..."
  push ecx
{ call &puts
```

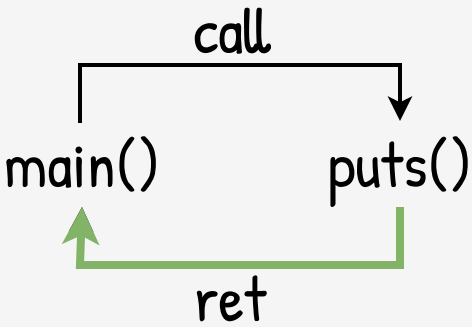


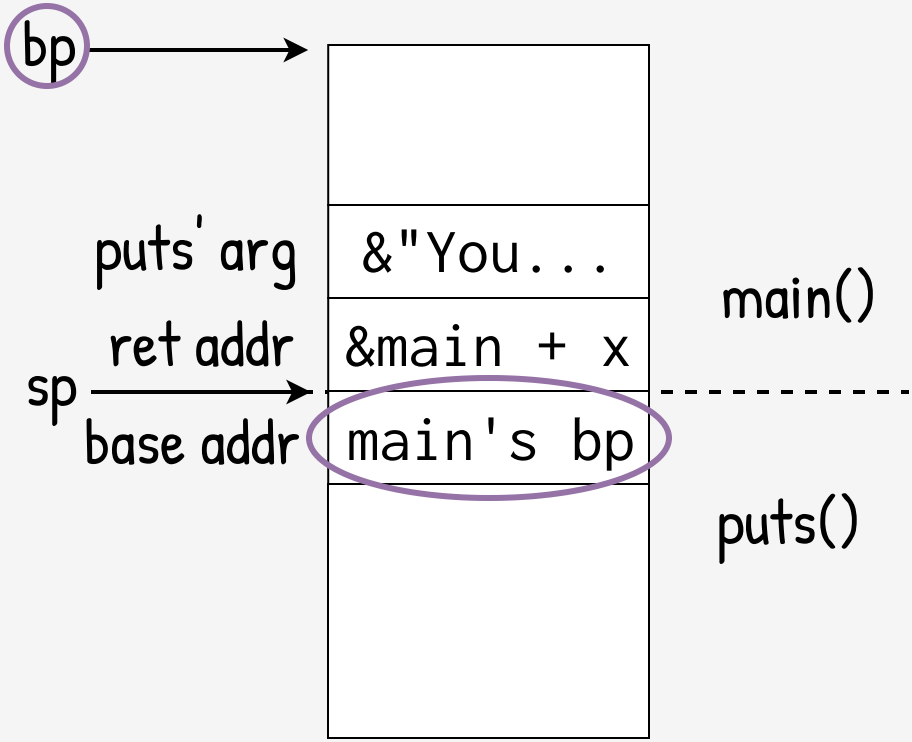



```
push ebp
mov ebp esp
mov ecx esp-8
```

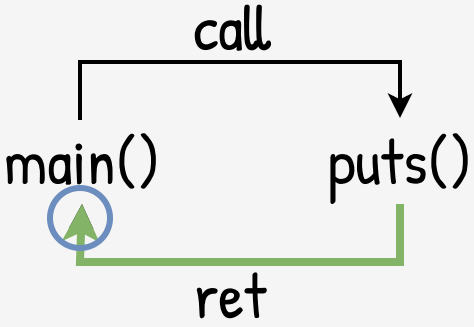

call
main()
puts()
ret

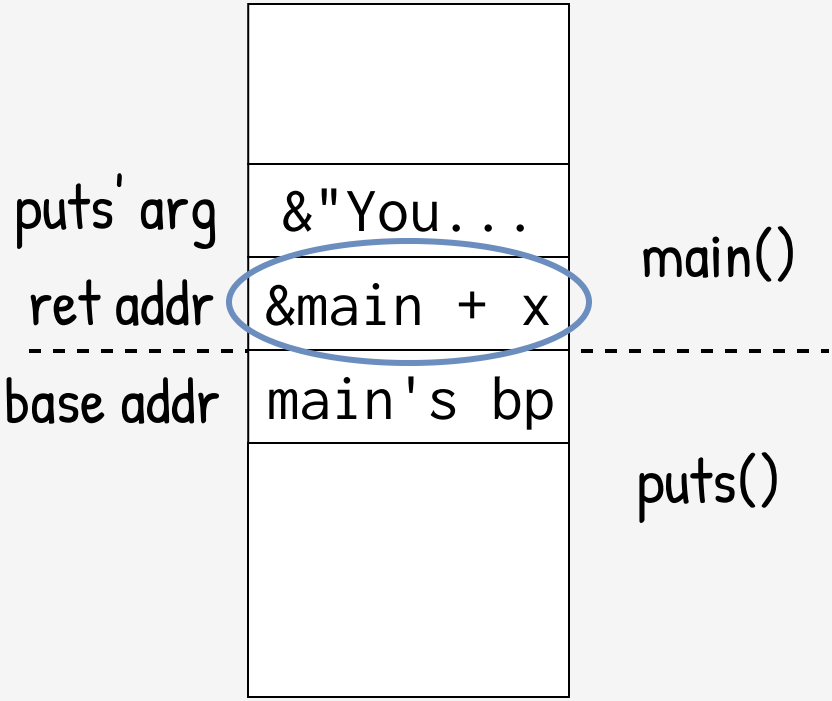

puts' arg
&"You...
ret addr
&main + x
main()
base addr
main's bp
puts()
sp


call
main()
puts()
ret

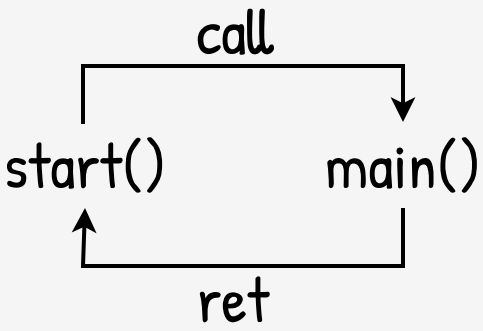

bp
puts' arg
&"You...
ret addr
&main + x
sp
base addr
main's bp
main()
puts()

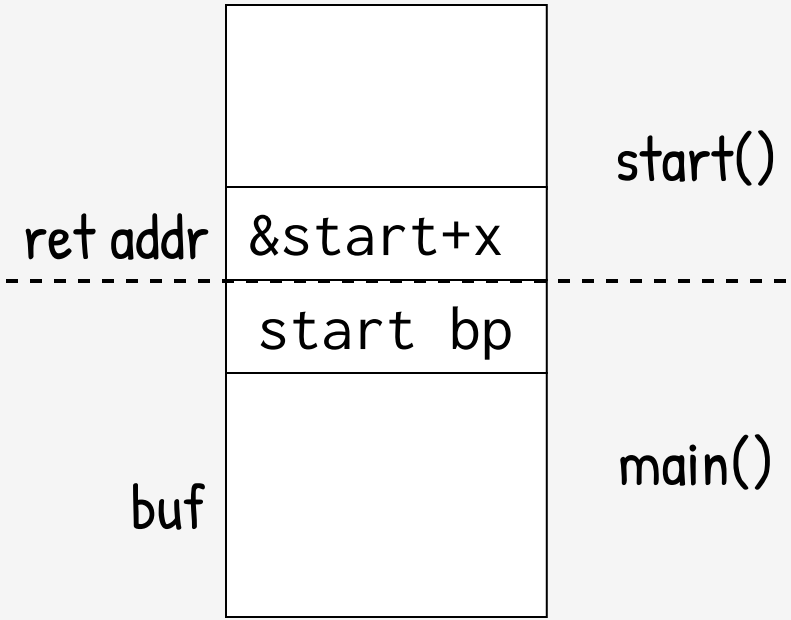
```
{ pop  ebp
  ret
```

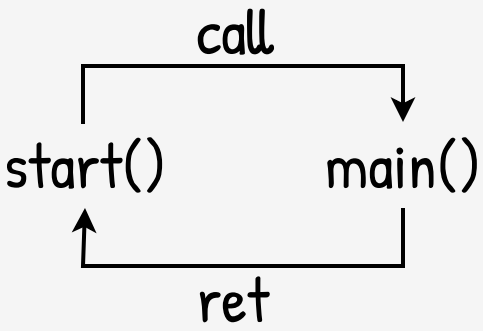

call
main()
puts()
ret

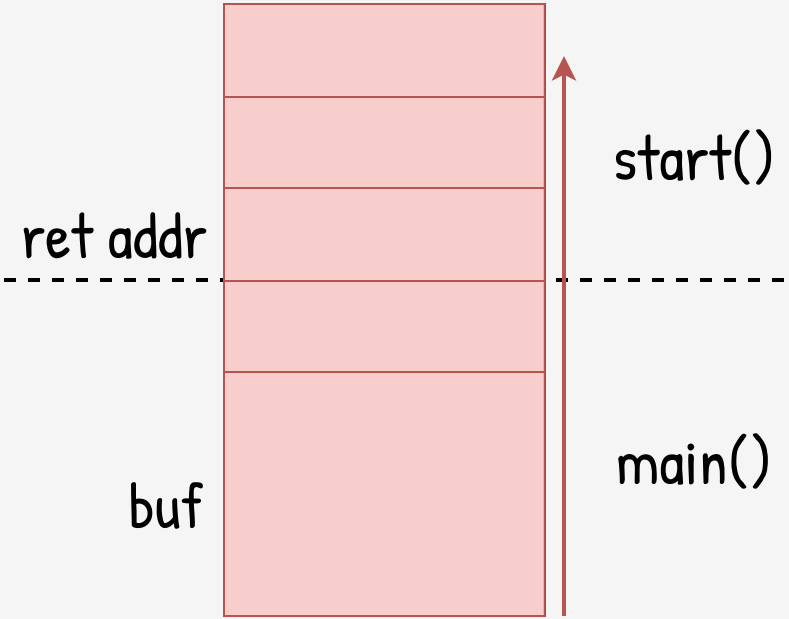

puts' arg
&"You...
ret addr
&main + x
main()
base addr
main's bp
puts()

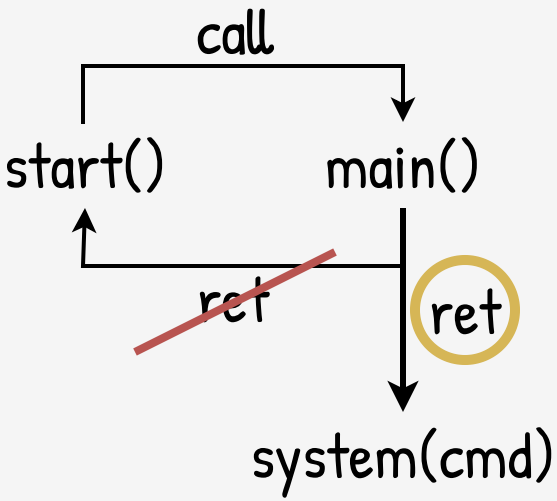
```
  pop  ebp
{ ret
```

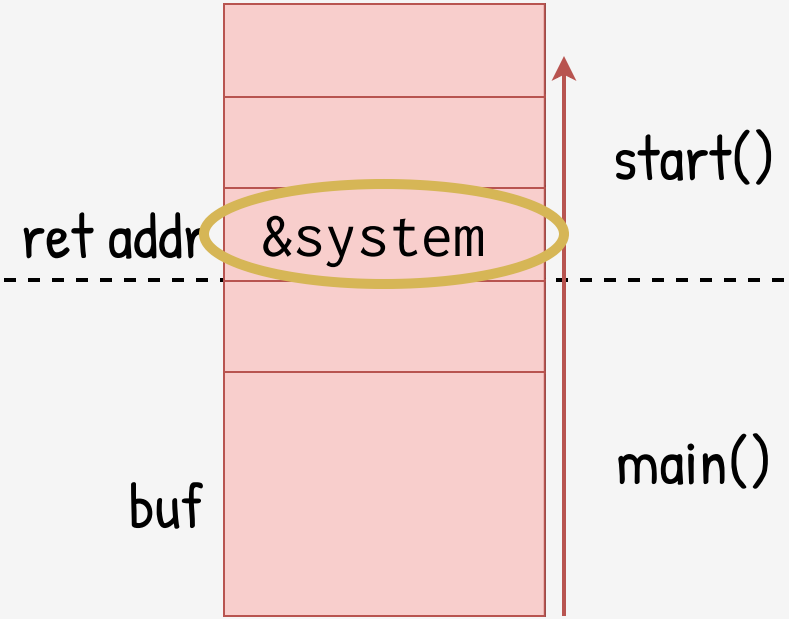

call
start()
main()
ret

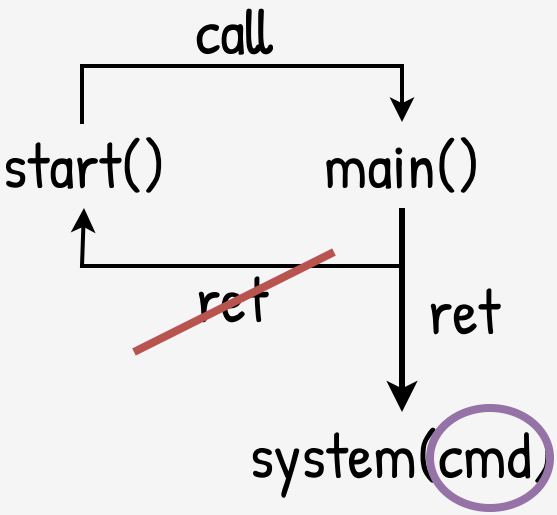

start()
ret addr
&start+x
start bp
main()
buf

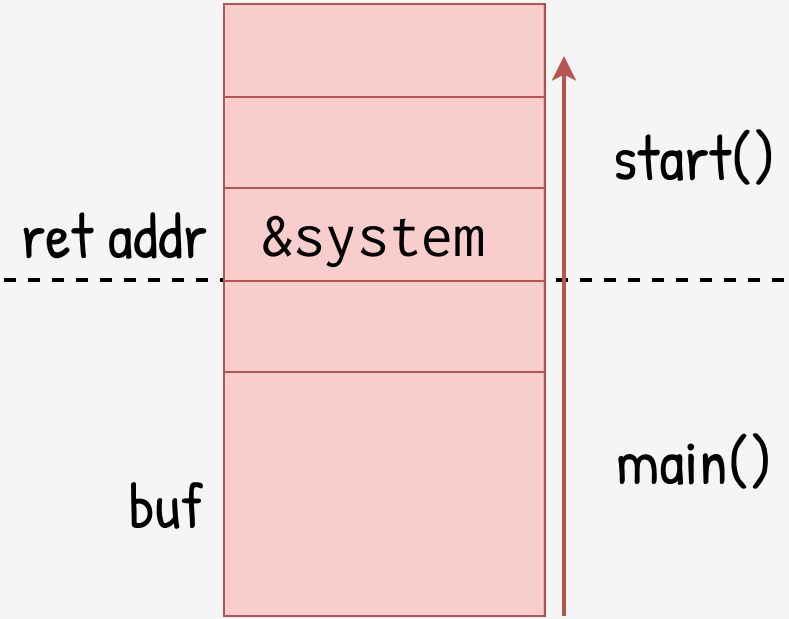

call
start()
main()
ret


start()
ret addr
main()
buf


call
start()
main()
ret
ret
system(cmd)


start()
ret addr
&system
main()
buf

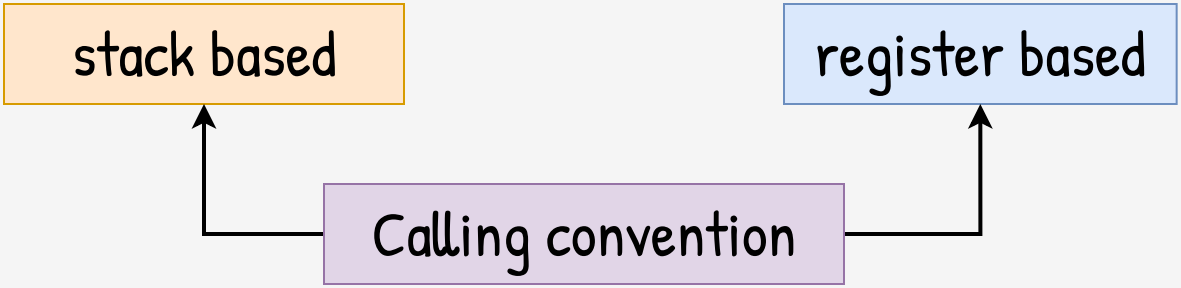

call
start()
main()
ret
ret
system(cmd)

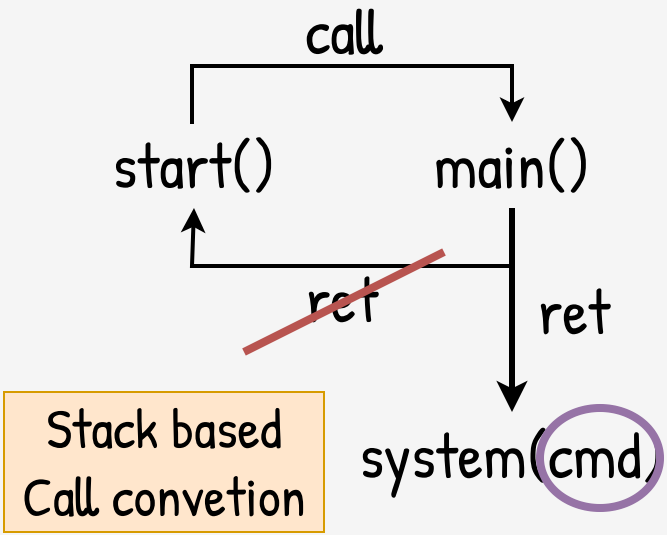

start()
ret addr
&system
main()
buf

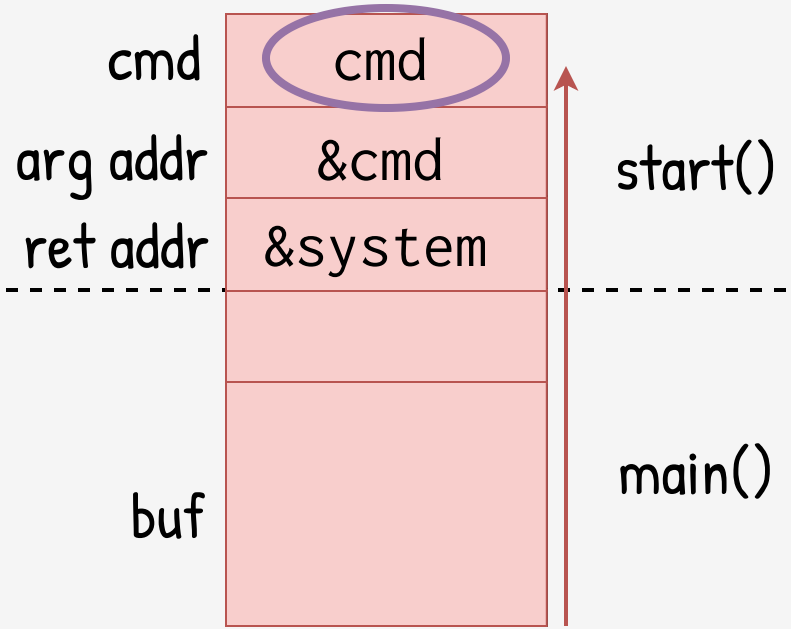
```
puts("Your flag is ~/flag.txt.");
```

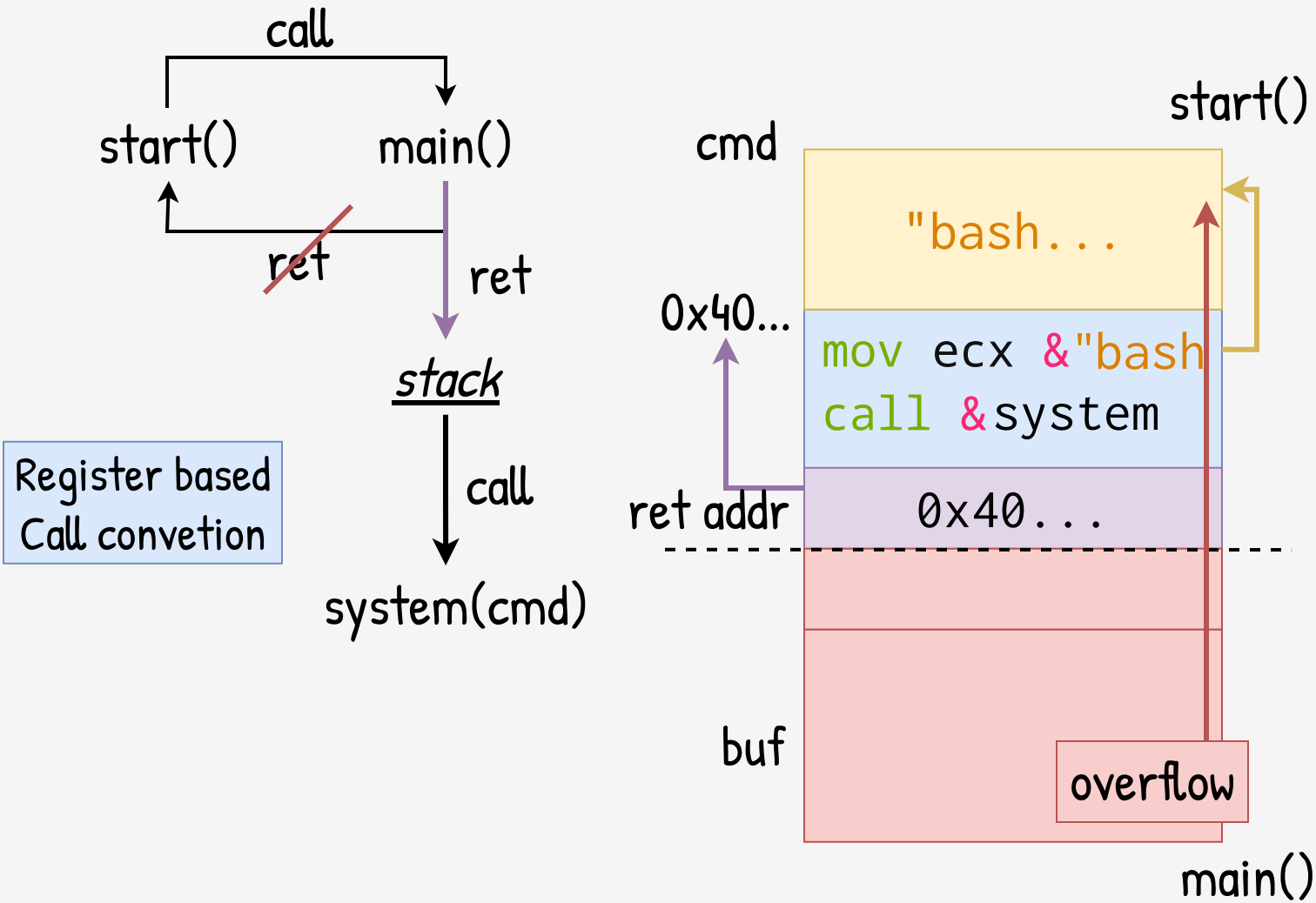
```
mov ecx &"You..."
push ecx
call &puts
```

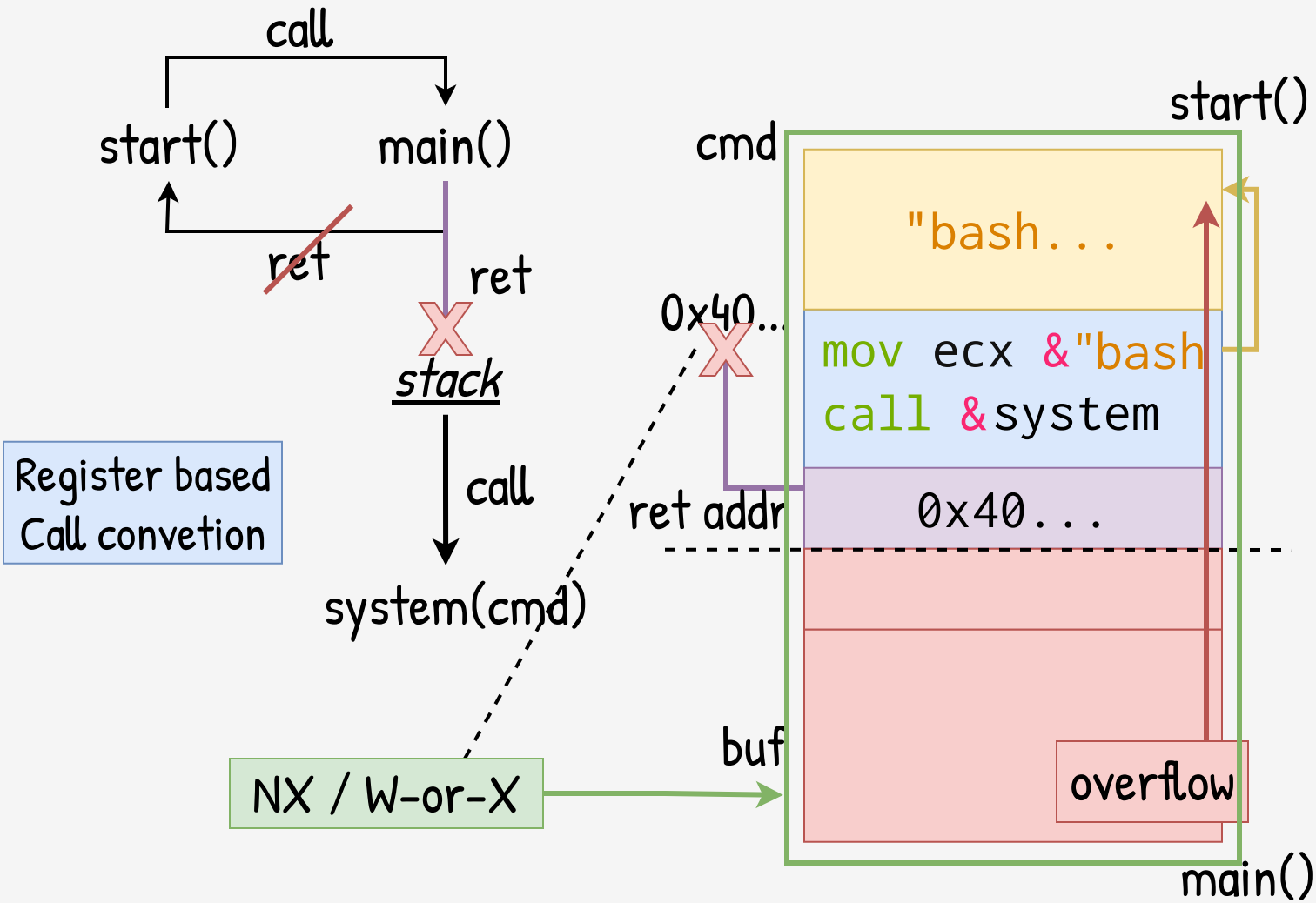
```
mov ecx &"You..."
call &puts
```

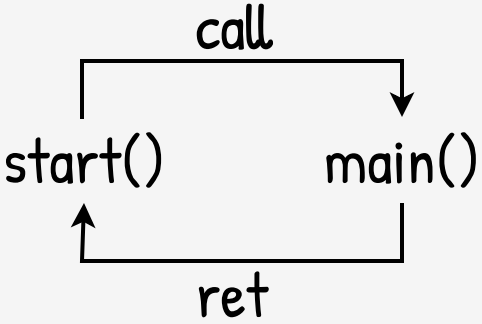



call
start()
main()
ret
ret
system(cmd)
Stack based
Call convetion


cmd
cmd
arg addr
&cmd
ret addr
&system
start()
buf
main()


call
start()
main()
ret
ret
stack
call
system(cmd)
Register based
Call convetion
start()
cmd
"bash...
0x40...
mov ecx &"bash
call &system
ret addr
0x40...
buf
overflow
main()

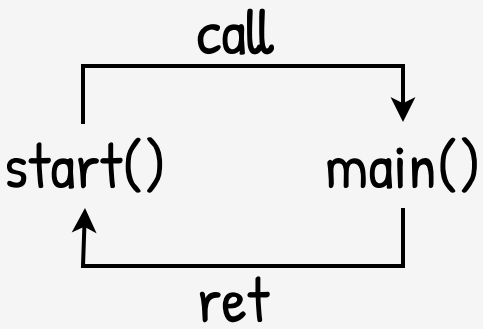

call
start()
main()
ret
ret
stack
call
system(cmd)
Register based
Call convetion
NX / W-or-X
cmd
start()
"bash...
0x40...
mov ecx &"bash
call &system
ret addr
0x40...
buf
overflow
main()



Goal:

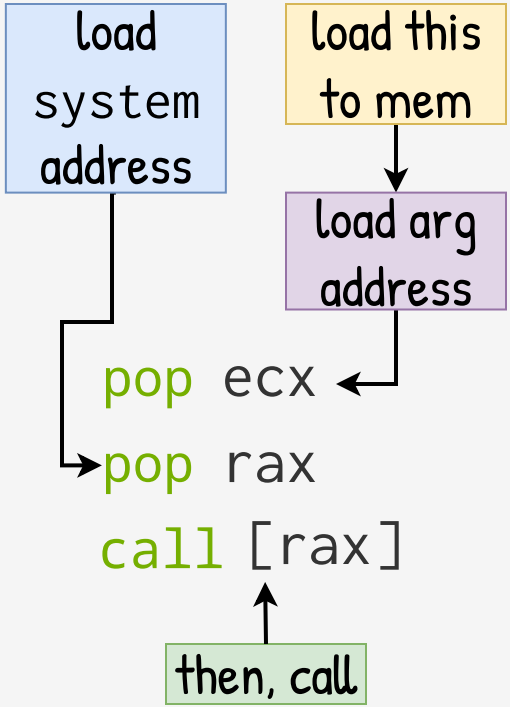
call `system` with

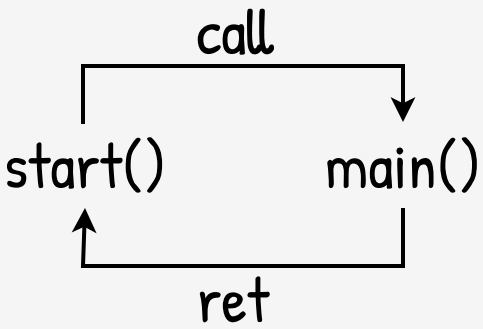
`"bash..."`



Goal:

call `system` with `"bash..."`



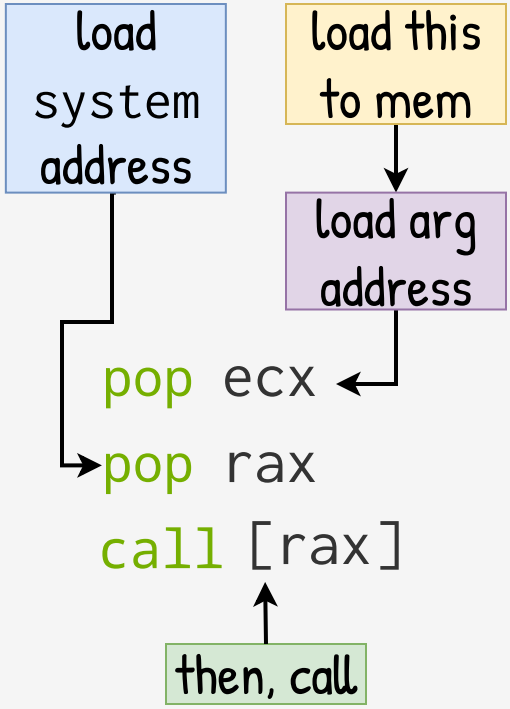


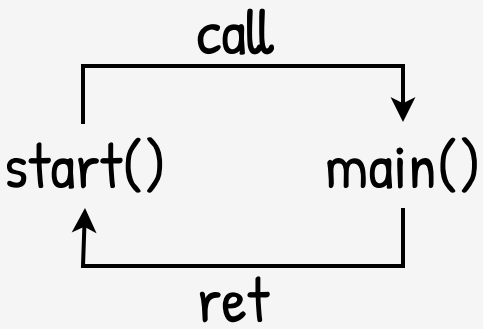

```
pop ecx
ret

pop rax
ret

call [rax]
```

From ro + executable program code in mem

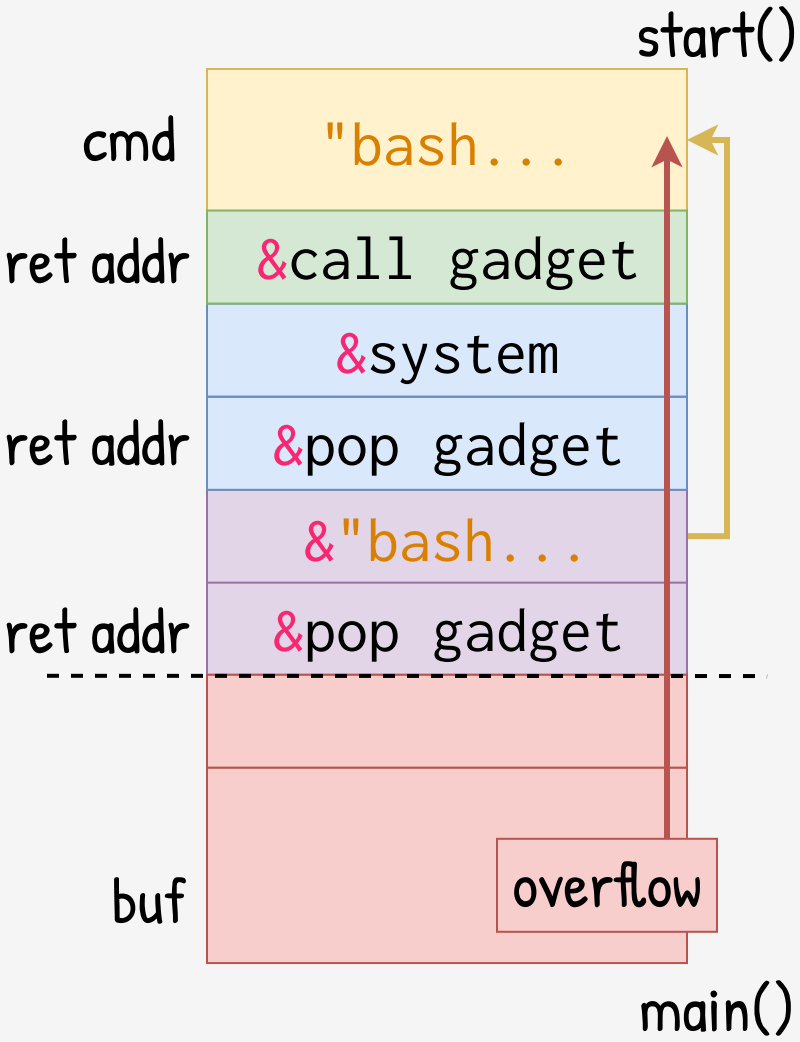
Goal:

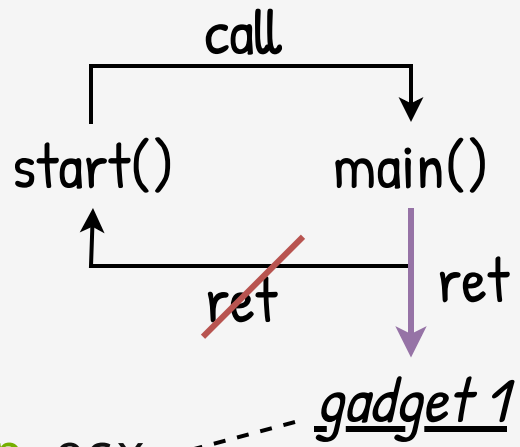
call `system` with `"bash..."`



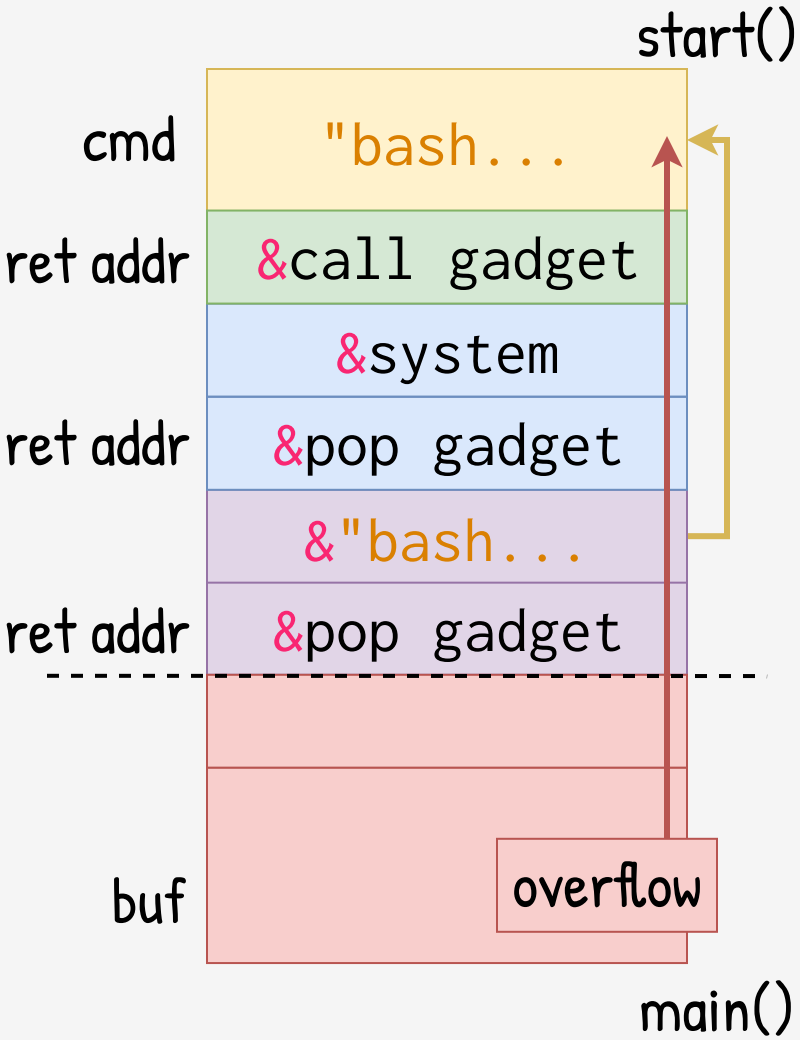


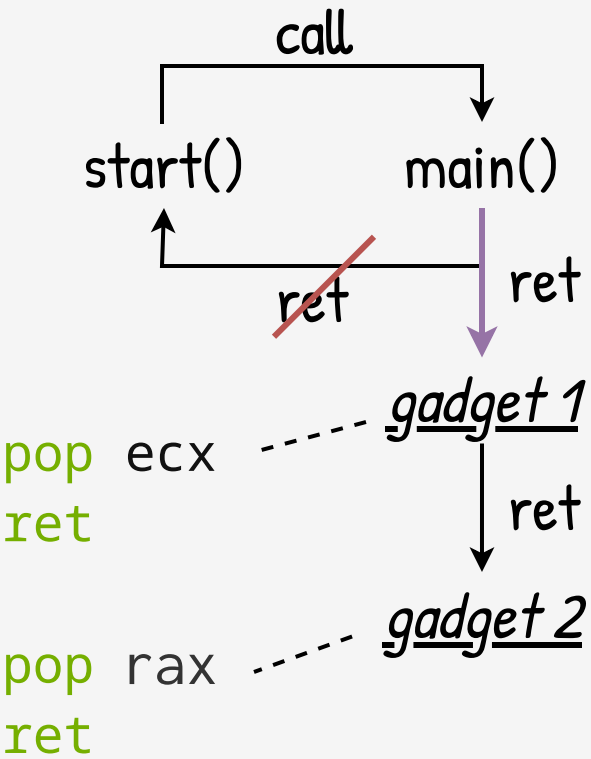

```
pop ecx
ret

pop rax
ret

call [rax]
```



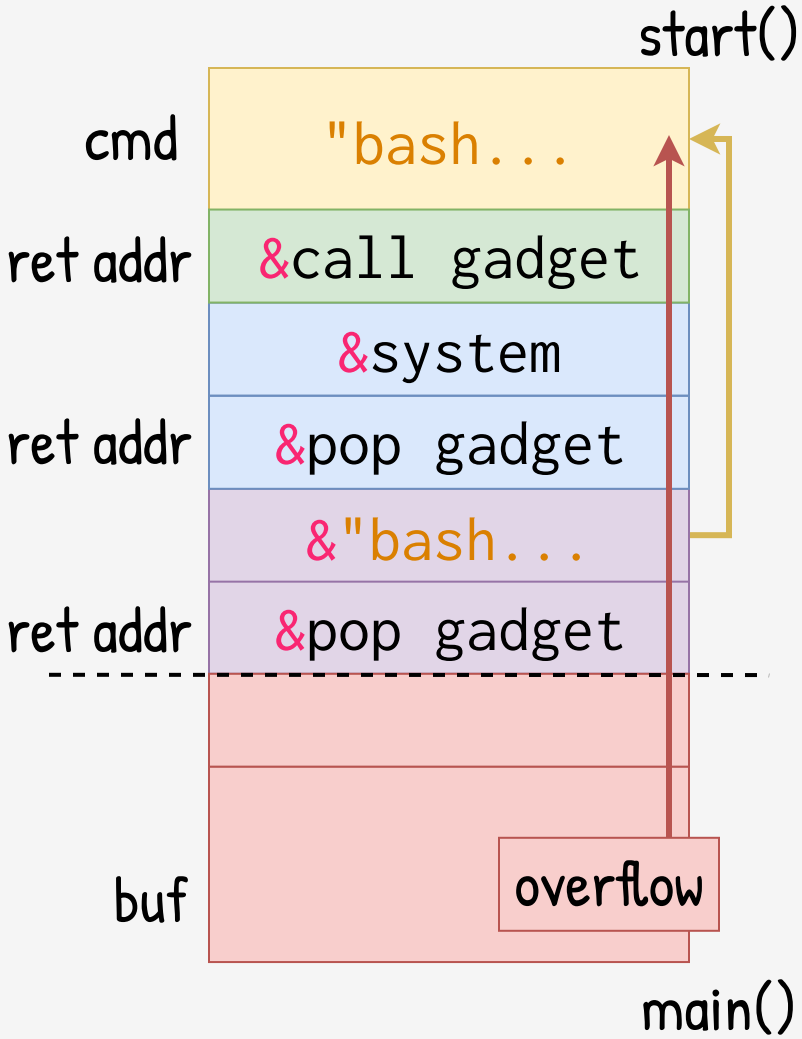


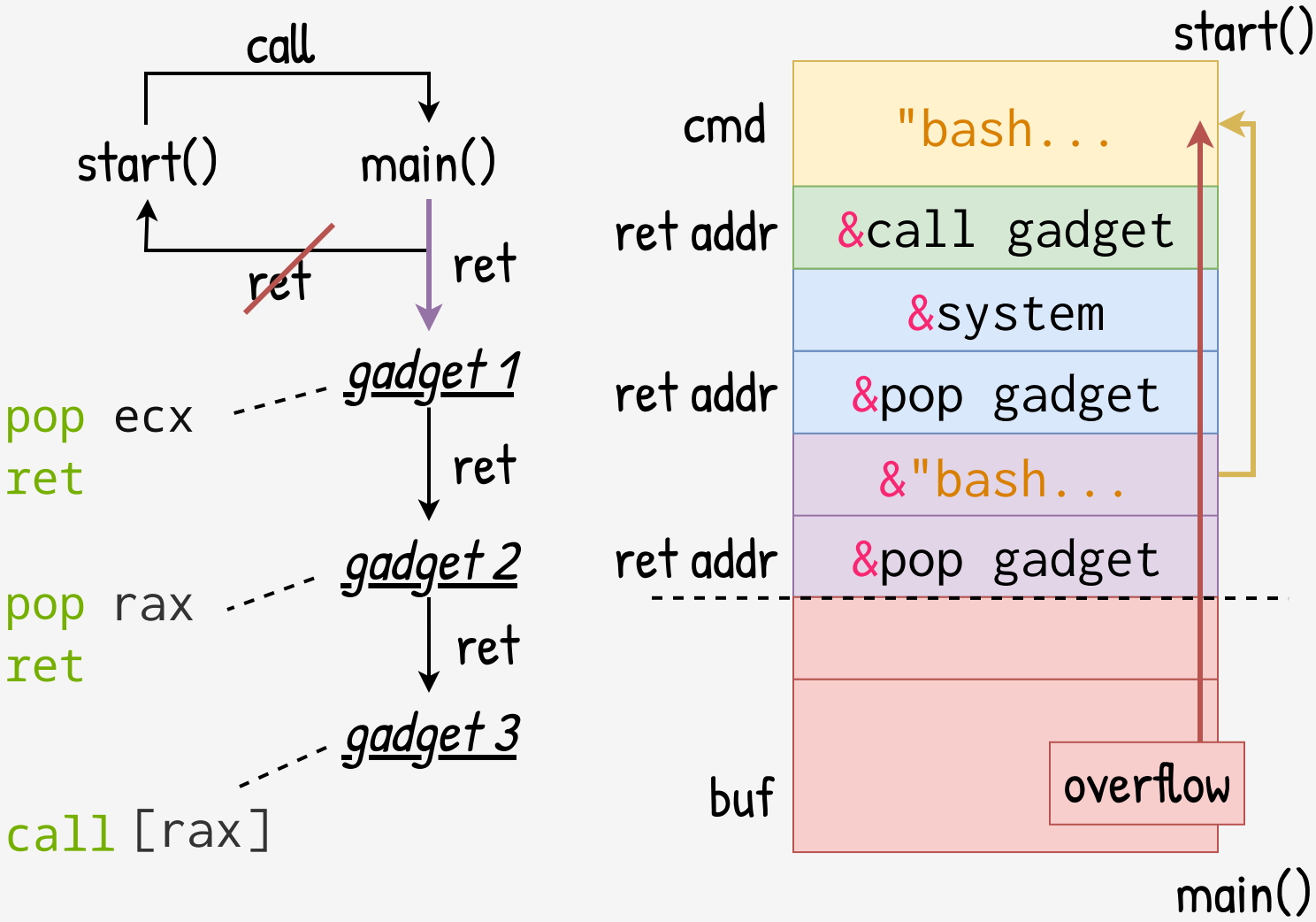

```
pop ecx
ret

pop rax
ret

call [rax]
```

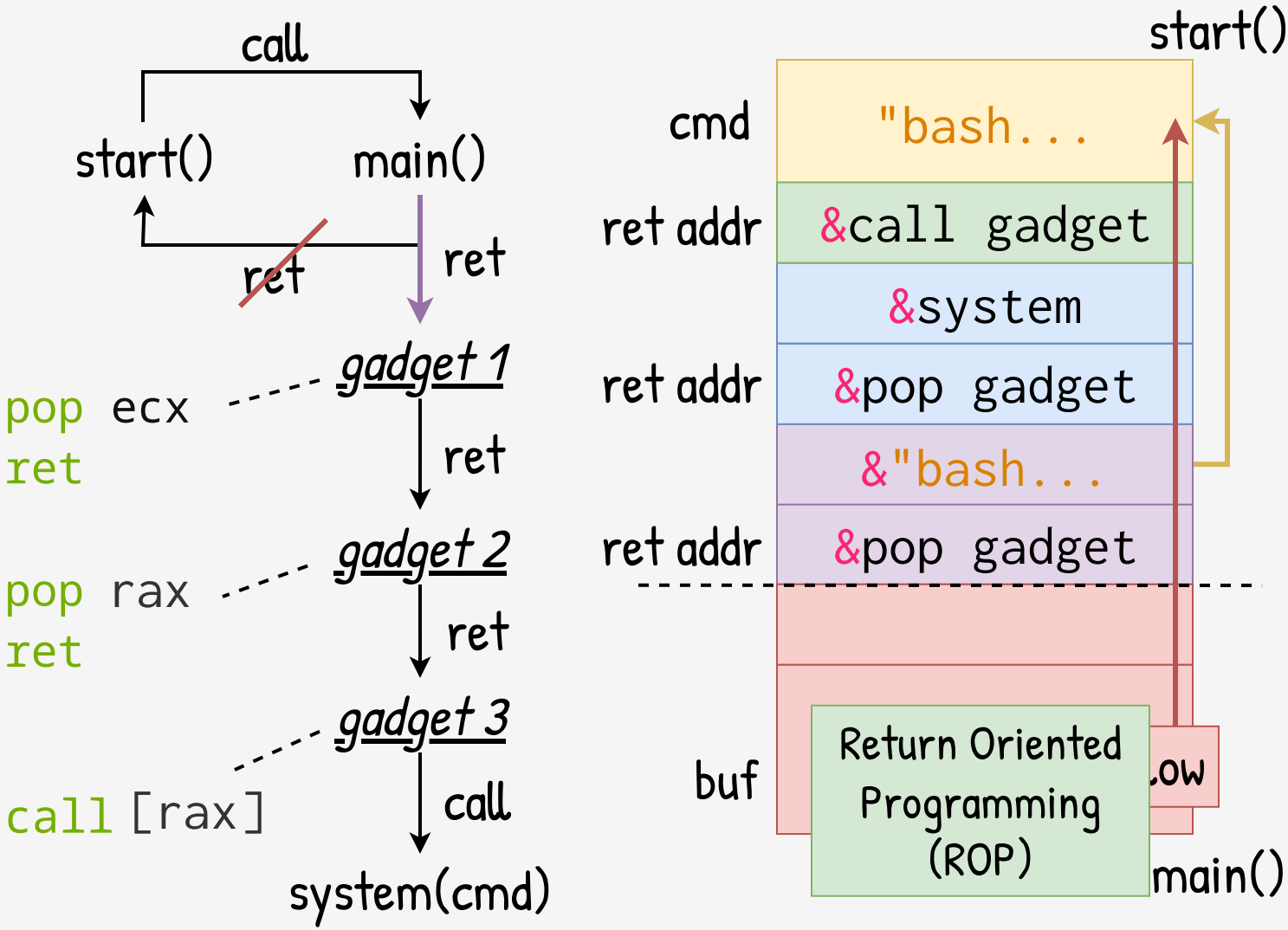


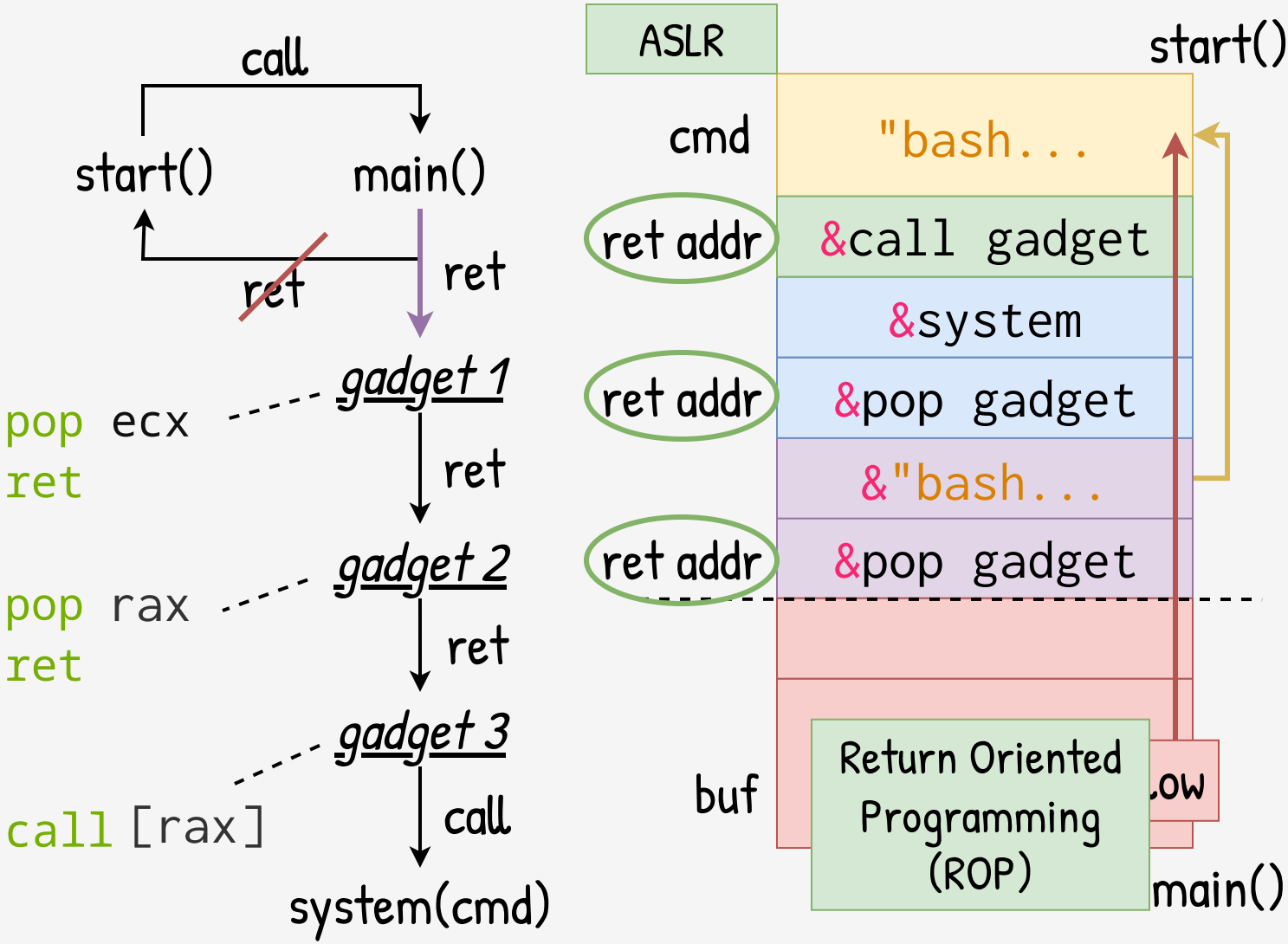

call
start()
main()
ret
ret
gadget 1
pop ecx
ret
ret
gadget 2
pop rax
ret

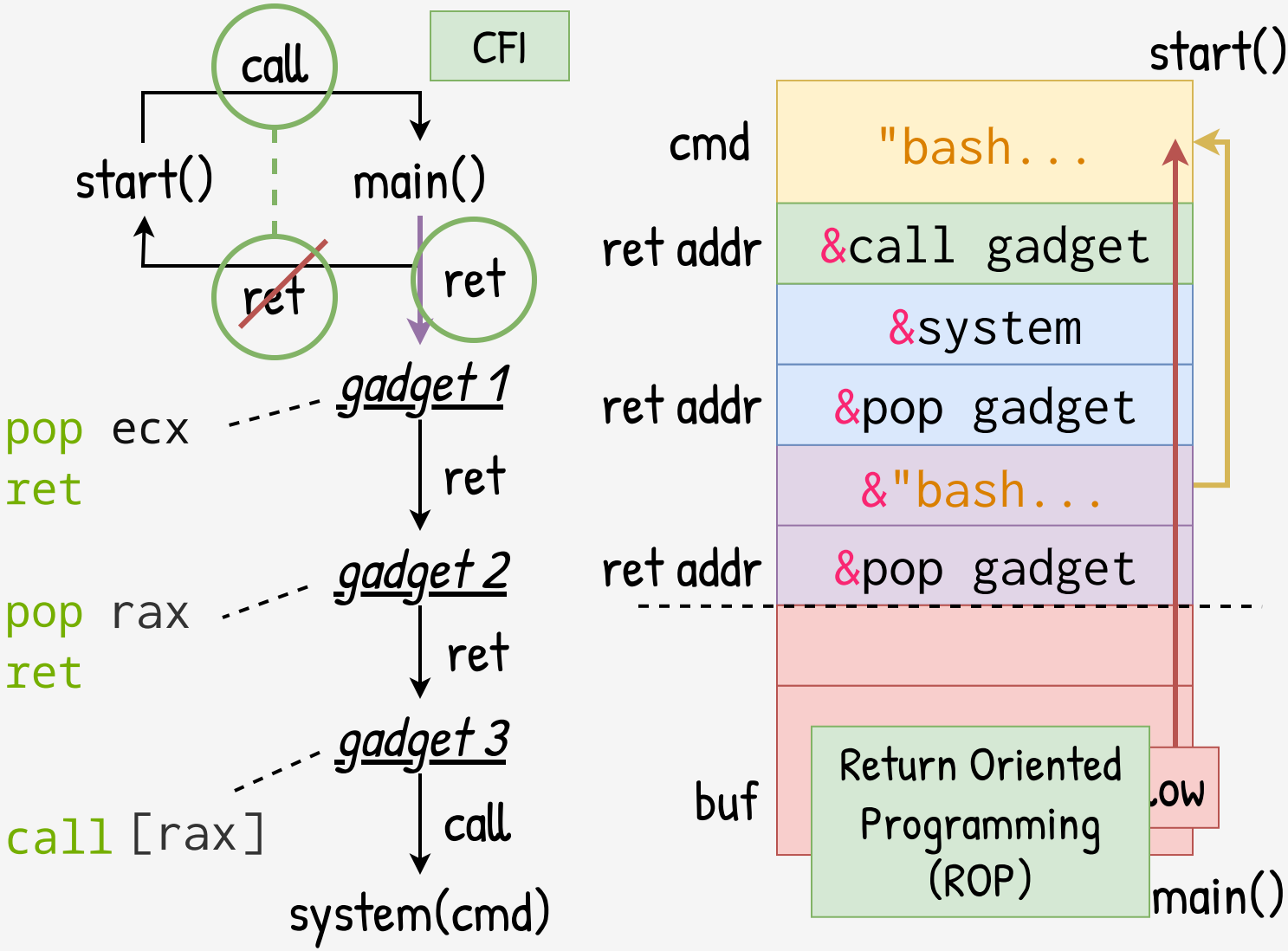
```
call [rax]
```

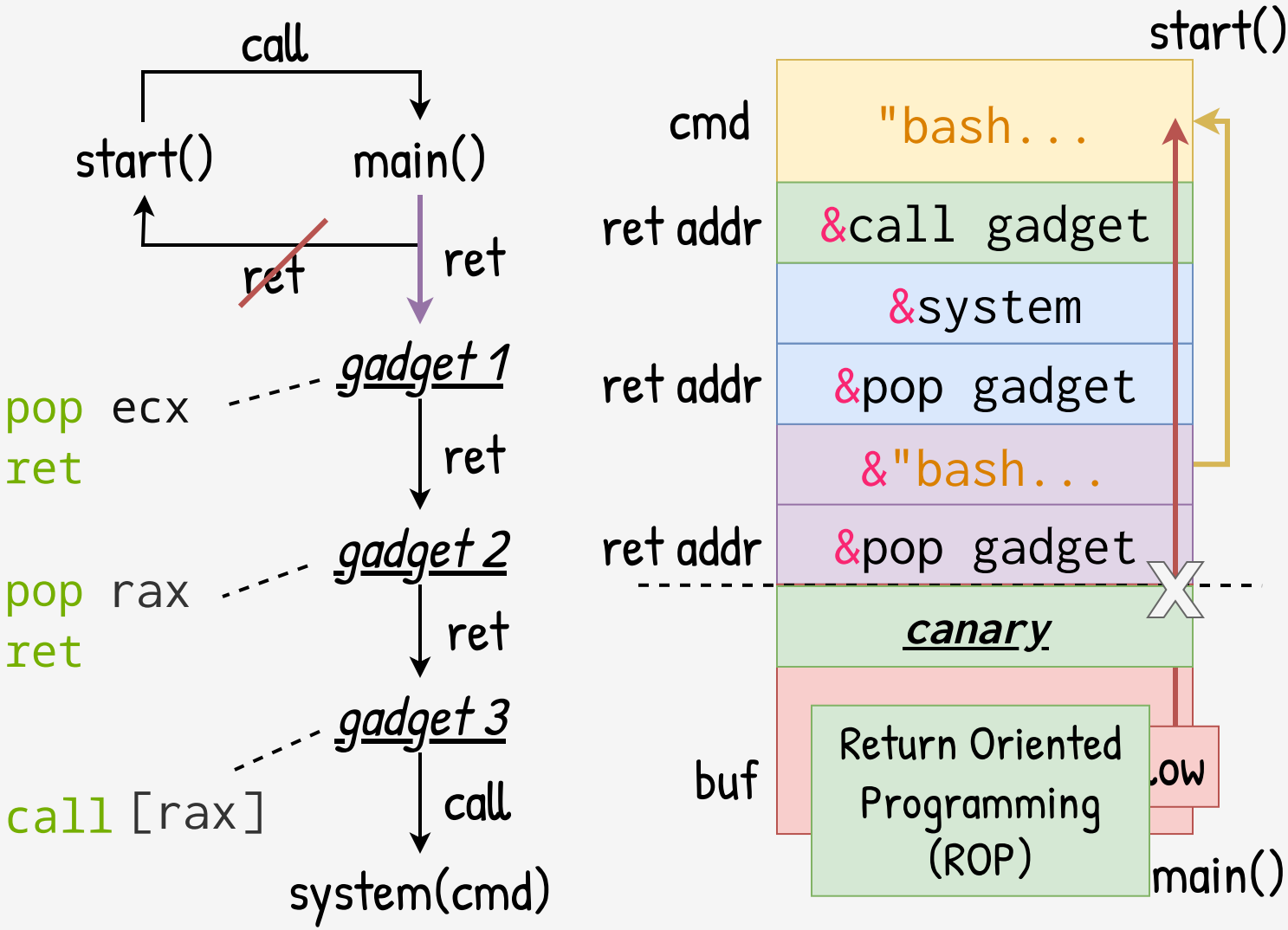

start()
cmd
"bash...
ret addr
&call gadget
&system
ret addr
&pop gadget
&"bash...
ret addr
&pop gadget
buf
overflow
main()


call
start()
main()
ret
ret
gadget 1
pop ecx
ret
ret
gadget 2
pop rax
ret
ret
gadget 3
call [rax]
start()
cmd
"bash...
ret addr
&call gadget
&system
ret addr
&pop gadget
&"bash...
ret addr
&pop gadget
buf
overflow
main()

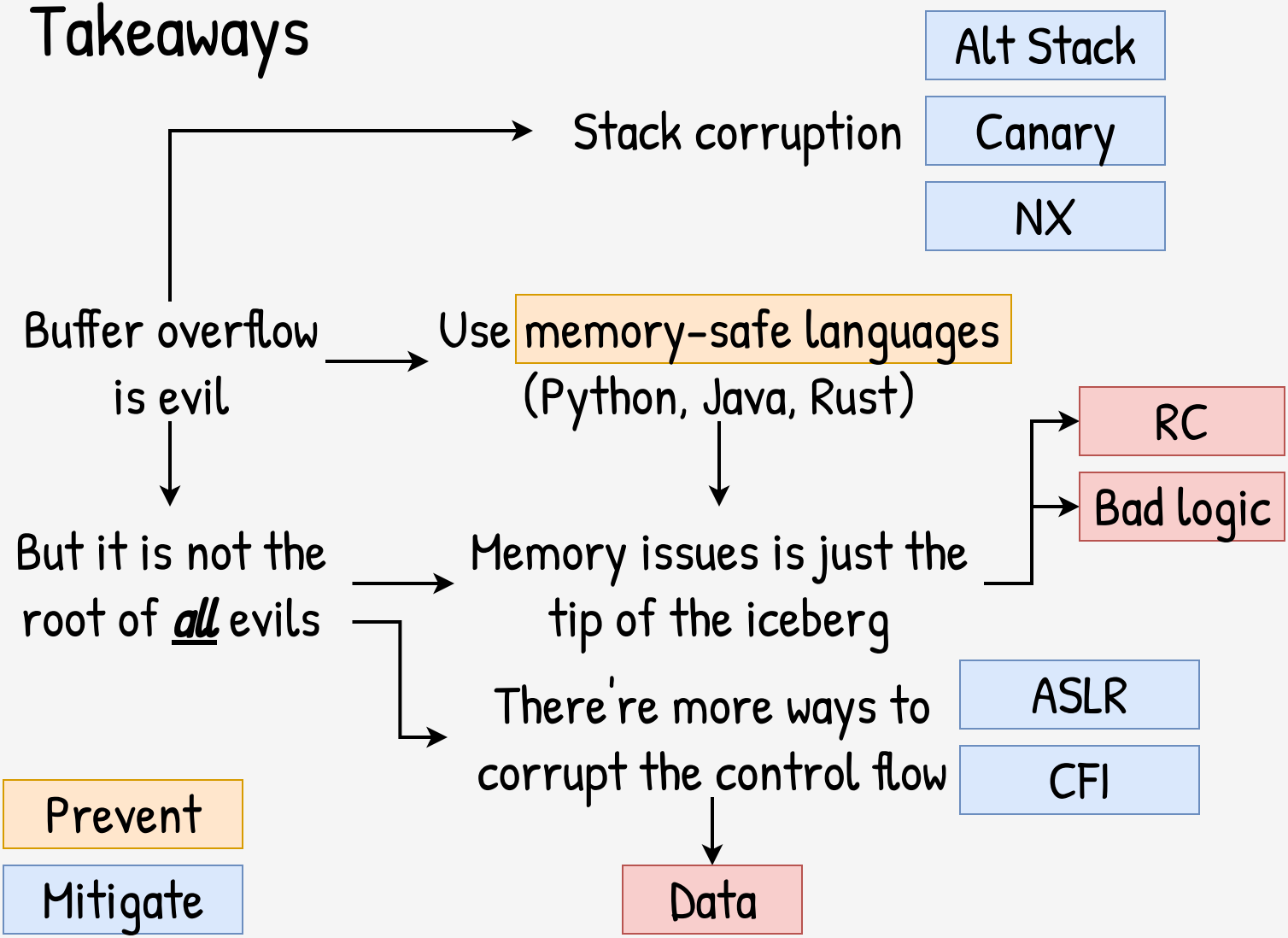

call
start()
main()
ret
ret
gadget 1
pop ecx
ret
ret
gadget 2
pop rax
ret
ret
gadget 3
call [rax]
call
system(cmd)
start()
cmd
"bash...
ret addr
&call gadget
&system
ret addr
&pop gadget
&"bash...
ret addr
&pop gadget
buf
Return Oriented Programming (ROP)
low
main()


call
start()
main()
ret
ret
gadget 1
pop ecx
ret
ret
gadget 2
pop rax
ret
ret
gadget 3
call [rax]
call
system(cmd)
ASLR
start()
cmd
"bash...
ret addr
&call gadget
&system
ret addr
&pop gadget
&"bash...
ret addr
&pop gadget
buf
Return Oriented Programming (ROP)
.ow
main()


CFI
call
start()
main()
ret
ret
gadget 1
pop ecx
ret
ret
gadget 2
pop rax
ret
ret
gadget 3
call [rax]
call
system(cmd)
start()
cmd
"bash...
ret addr
&call gadget
&system
ret addr
&pop gadget
&"bash...
ret addr
&pop gadget
buf
Return Oriented Programming (ROP)
.ow
main()


call
start()
main()
ret
ret
gadget 1
pop ecx
ret
ret
gadget 2
pop rax
ret
ret
gadget 3
call [rax]
call
system(cmd)
start()
cmd
"bash...
ret addr
&call gadget
&system
ret addr
&pop gadget
&"bash...
ret addr
&pop gadget
canary
buf
Return Oriented Programming (ROP)
.ow
main()

Takeaways


Alt Stack
Stack corruption
Canary
NX
Buffer overflow is evil
Use memory-safe languages (Python, Java, Rust)
RC
Bad logic
But it is not the root of all evils
Memory issues is just the tip of the iceberg
There're more ways to corrupt the control flow
ASLR
CFI
Prevent
Mitigate
Data